

Relatório Técnico da Sprint 4

Sistema de Hospedagem | Grupo 8 | 29 de junho de 2026

Nas sprints anteriores a gente construiu o sistema de hospedagem com as classes de domínio, a API, o banco de dados, os testes e o tratamento de erros. Nesta etapa o objetivo foi melhorar a arquitetura usando padrões de projeto. Escolhemos duas funcionalidades novas e aplicamos um padrão em cada uma, e ainda usamos o Singleton como um recurso global do sistema.

As funcionalidades que escolhemos

Escolhemos a Tarifação Flexível e a Montagem de Pacotes de Hospedagem. As duas combinam bem com o que o sistema já fazia, porque as duas mexem no preço da estadia, e cada uma deixa clara a ideia de um padrão diferente. Com isso a gente conseguiu mostrar dois padrões bem conhecidos trabalhando dentro do mesmo projeto.

Tarifação flexível com o padrão Strategy

O problema: o preço da diária precisava mudar conforme a situação, como alta temporada, baixa temporada ou uma promoção. Se a gente jogasse vários if dentro do cálculo, o código ia ficar grande e chato de mexer toda vez que aparecesse uma regra nova.

A solução: usamos o padrão Strategy. Criamos uma interface chamada Tarifa, com o método aplicar, e cada regra virou uma classe separada: TarifaNormal, TarifaAltaTemporada, TarifaBaixaTemporada e TarifaPromocional. Na hora de fechar um aluguel, o sistema pega a tarifa que está ativa e aplica sobre o valor da diária. Para criar uma regra nova, basta escrever uma classe nova que implementa Tarifa, sem precisar abrir o que já existe.

Pacotes de hospedagem com o padrão Decorator

O problema: além da hospedagem, o cliente pode contratar serviços como café da manhã, passeio, transporte e lavanderia, e esses serviços podem ser combinados de qualquer jeito. Criar uma classe fixa para cada combinação possível geraria um número enorme de classes.

A solução: usamos o padrão Decorator. A Hospedagem é o serviço base, com um preço. Cada serviço adicional envolve um serviço que já existe e soma o seu valor e o seu nome na descrição. Assim dá para montar hospedagem mais café, ou hospedagem mais café mais passeio, do jeito que o cliente quiser. Para oferecer um serviço novo, a gente cria uma classe que estende ServicoAdicional, e o resto continua igual.

O Singleton GerenciadorTarifas

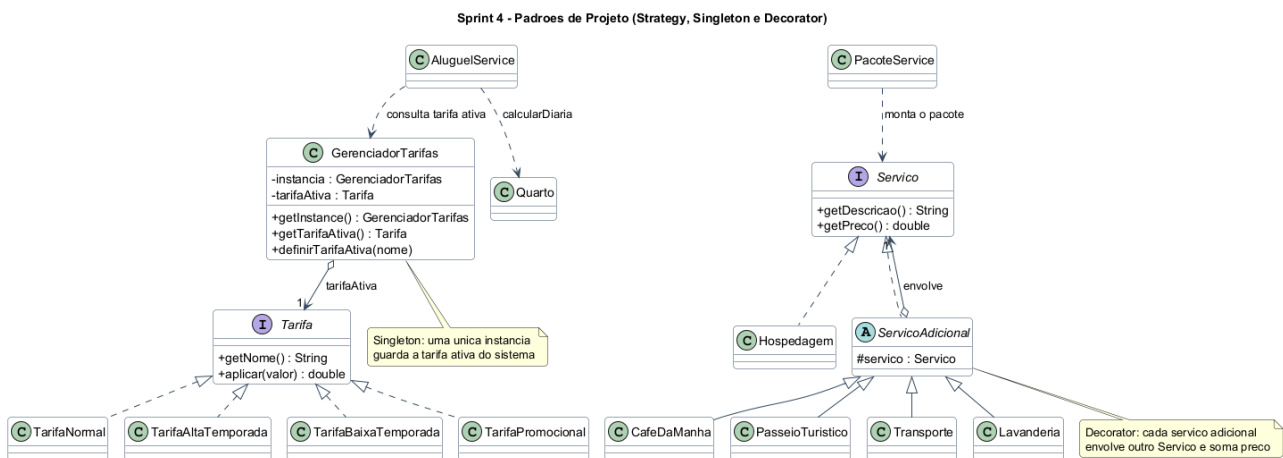
A tarifa ativa é uma informação que vale para o sistema inteiro: todos os aluguéis precisam usar a mesma política de preço no mesmo momento. Por isso ela precisa ficar guardada em um lugar só. Criamos o GerenciadorTarifas como Singleton, que é uma classe com uma única instância. O construtor dele é privado e o acesso acontece pelo método getInstance, que sempre devolve a mesma instância. Ele guarda a lista de tarifas e qual delas está ativa.

A justificativa é simples: se cada parte do sistema criasse o seu próprio gerenciador, dois aluguéis poderiam acabar usando tarifas diferentes sem querer, e foi exatamente isso que a gente quis evitar. Vale lembrar que o Singleton não está sozinho. Ele trabalha junto com o

Strategy, que é quem de fato calcula o preço, então ele é só o ponto único que guarda a configuração.

Como a gente testou

Para mostrar que tudo funciona, escrevemos testes automáticos com JUnit. Os testes de tarifa conferem que a normal mantém o valor, a alta multiplica por 1,5, a baixa por 0,8 e a promocional por 0,9, e que o gerenciador devolve sempre a mesma instância e troca a tarifa ativa direitinho. Os testes de pacote conferem que a hospedagem sozinha custa o valor base, que o café soma 30, e que empilhando hospedagem, café, passeio e transporte o total fecha em 730. Somando com os testes que já existiam, o projeto tem 25 testes e todos passam. O diagrama abaixo mostra como as classes novas se organizam e como elas conversam com as classes que já existiam.



Os benefícios da nova arquitetura

Com essa arquitetura o sistema ficou bem mais fácil de crescer. Para uma nova regra de preço ou um novo serviço, a gente só adiciona uma classe e pronto, sem mexer no código antigo e sem o risco de quebrar o que já estava funcionando. O preço da estadia continua sendo calculado em um lugar claro, e a tarifa que vale para todo mundo fica guardada em um ponto único. No fim das contas, é menos código repetido, mais organização e bastante espaço para o sistema evoluir nas próximas necessidades.